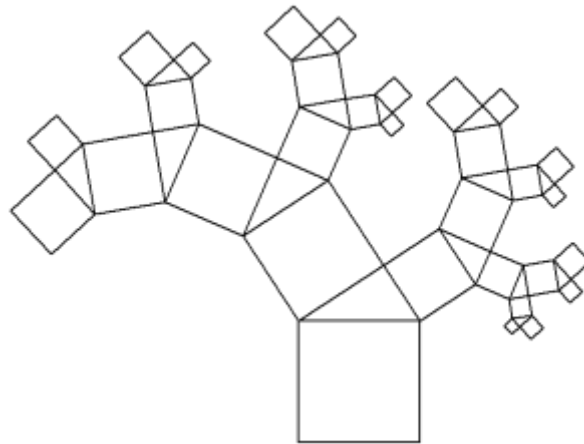


Pythagoras-Baum in 3D

Projekt 12



Projektbeschreibung und Ziele

Der Pythagoras-Baum ist ein rekursives Fraktal, das ursprünglich in 2D aus rechtwinkligen Quadraten erzeugt wird. In diesem Projekt soll das Team die Idee in den dreidimensionalen Raum übertragen und einen 3D-Fraktalbaum erzeugen, der sich visuell an Pflanzenwachstum anlehnt und gleichzeitig strukturierte Parallelisierung erlaubt.

Die Basisgeometrie besteht aus Quadern, die ähnlich dem 2D-Ansatz, stetig wächst. Die Quader setzen immer auf einen Parent-Quader auf und die Winkel zueinander können variieren.

Das Ziel dieses Projekts ist die Generierung eines Dreieck-Meshs als Pythagoras-Baum. Jeder Quader besteht aus festen 12 Dreiecken, jede Seite der Quader aus 2 Dreiecken. Diese können anschließend mit OpenGL gerendert werden.

Ziele

Implementieren Sie die folgenden Funktionalitäten in Ihrem Projekt. Setzen Sie diese sowohl sequentiell, als auch parallel um.

1. Implementierung des Pythagoras Baums in 3D mit Quadern statt Quadrate
2. Generierung eines Mesh aus Dreiecken durch die Quader
3. Randomisierung des Baumes durch zufällige Variation der Winkel zwischen den Kind-Quadern
4. Abbruchbedingung bei einer bestimmten Asttiefe
5. Rendering mit OpenGL
6. Optional: Level-Of-Detail durch Distanz zur Kamera

Variation der Baumtiefe sollen die Rechenintensität erhöhen.

Technologien

Nutzen Sie für Ihr Projekt folgende Technologien:

- **OpenMP** (Multithreading auf der CPU)
- **MPI** (Verteilung der Arbeit auf mehrere Prozesse und Nodes)
- **CUDA** oder **OpenCL** (Wählen Sie eine der beiden Technologien aus)

Das Ziel beim Einsatz der verschiedenen Technologien ist es, die kürzeste Laufzeit gegenüber der sequentiellen Implementierung zu erreichen. Dieser Speedup kann durch den Einsatz einzelner Technologien oder durch eine Kombination derselben erzielt werden. Da jede Technologie unterschiedliche Stärken und Schwächen besitzt, müssen Sie für jede Operation analysieren und begründen, welche Technologie geeignet ist und welche Vorteile oder Nachteile sich daraus ergeben. Sollte sich eine Technologie aus Ihrer Sicht überhaupt nicht für eine Funktionalität eignen, muss diese Entscheidung in der Dokumentation begründet werden.

Leistungsanalyse

Erstellen Sie eine aussagekräftige Analyse der Laufzeiten und Skalierbarkeit aller implementierten Funktionen. Ihre Analyse sollte beinhalten:

- Messungen verschiedener Dimensionen
- Vergleiche zwischen *sequentiell* - *OpenMP* - *MPI* - *GPU*
- Variationen der Parameter (Anzahl Quader, Baumtiefe)
- Diagramme erstellen, z.B. Laufzeit für verschiedene Parameterwerte, Speedup-Kurven, etc.
- Interpretation der Messergebnisse
- Erklärungen, wie die Speicherhierarchie das Verhalten beeinflusst

Dokumentation

Erstellen Sie eine Projekt-Dokumentation im Ordner **doc/** als **PDF**:

- Überblick über das Projekt, Architektur und Aufbau
- Beschreibung der Implementierungsentscheidungen: "Was wurde wann, warum gemacht?"
- Hinweise zur Parallelisierung (Granularität, Grade, Operationen, Muster)
- Verwendete Datenstruktur und Hinweis auf die Caches
- Anleitung zum Bauen und Ausführen des Projekts
- Leistungsanalyse mit Laufzeitdiagrammen und Interpretationen
- Begründung zu den verwendeten Technologien
- Möglichst viele visuelle Ergebnisse, sofern möglich
- Beschreibung zusätzlicher Libraries, sofern verwendet
 - exakte Version und Downloadquelle
 - Installationshinweise

Abgabe

- **Deadline** 06.02.2026, 23:59 Uhr.
 - Es zählt der Zeitstempel des letzten Commits auf dem **main**-Branch in Github.
- **Projektdateien**
 - Legen Sie alle *.sln* und *.vcxproj* Dateien bei.
 - Nutzen Sie primär Visual Studio und den MSVC-Compiler, es sei denn, Sie haben zuvor eine andere Umgebung abgesprochen.
- **Funktionalität**
 - Alle Funktionalitäten müssen vollständig und korrekt implementiert sein. Die Umsetzung muss sequentiell und mit mindestens einer parallelen Technologie erfolgt sein.
- **Dokumentation**
 - Muss als PDF im **doc/** Ordner liegen
 - Andere Formate werden nicht akzeptiert